

DriveEase

Data Layer

Database access, seeding, and the in-memory store
ASP.NET Core 9.0 MVC | Entity Framework Core | PostgreSQL (Supabase)

What the Data layer does

The Data layer is everything that reads and writes persistent state. DriveEase currently has **two data mechanisms side by side**, reflecting a phased build: an Entity Framework Core **DbContext** backed by PostgreSQL (the real database), and a thread-safe **in-memory store** that the business services actually use today. Both are seeded with the same demo data on startup.

Files in this layer

File	Role	Lifetime
ApplicationDbContext.cs	EF Core DbContext: DbSets + relationship config for Postgres.	Scoped
DbSeeder.cs	Seeds the Postgres database on startup if empty.	Static
IDataStore.cs	Interface abstracting the in-memory store (Phase 1).	Singleton
InMemoryDataStore.cs	Thread-safe in-memory lists + seed data used by services.	Singleton

1. ApplicationDbContext

Inherits DbContext and exposes one **DbSet** per entity. A DbSet is a queryable/updatable collection that EF Core maps to a table.

DbSets (tables)

```
public DbSet<User> Users => Set<User>();  
public DbSet<Vehicle> Vehicles => Set<Vehicle>();  
public DbSet<Booking> Bookings => Set<Booking>();  
public DbSet<Review> Reviews => Set<Review>();  
public DbSet<Payment> Payments => Set<Payment>();  
public DbSet<Notification> Notifications => Set<Notification>();  
public DbSet<PromoCode> PromoCodes => Set<PromoCode>();  
public DbSet<PricingRule> PricingRules => Set<PricingRule>();
```

OnModelCreating - schema rules

Configuration	Applied to	Effect
Unique index	User.Email, Vehicle.LicensePlate, Booking.BookingReference, PromoCode.Code	DB rejects duplicate values.
Booking -> User / Vehicle	FK UserId / VehicleId	Restrict on delete (protects history).
Booking -> PromoCode	FK PromoCodeId (nullable)	SetNull on delete.
Payment -> Booking	1-to-1	Cascade delete.
Review -> Booking	1-to-1	Cascade delete.
Notification -> User	FK UserId	Cascade delete.

2. DbSeeder

A static class with `SeedAsync(ApplicationDbContext)`, called once from `Program.cs` at startup. It first checks `if (await context.Users.AnyAsync()) return;` so it never double-seeds. It then inserts, saving in stages so generated ids are available for foreign keys:

Step	Seeds	Notes
1	3 Users	1 admin + 2 customers (Rahul, Priya).
2	8 Vehicles	Bikes, hatchback, sedan, SUVs, luxury; one under maintenance.
3	3 PromoCodes	WELCOME20, FLAT500, EXPIRED50 (expired for testing).
4	1 PricingRule	Weekend surge x1.15 on Saturday.
5	Completed booking	Priya + Honda City, with Payment and Review.
6	Future booking	Rahul + Swift, Confirmed, with Payment.
7	1 Notification	Welcome SMS to Rahul.

Because the DB generates identity ids, the seeder calls `SaveChangesAsync()` after users and after vehicles so their ids exist before bookings reference them.

3. IDataSource

An interface that abstracts the in-memory store so services do not depend on a concrete class. It exposes one `List<T>` per entity, an id-sequence method per entity (e.g. `NextUserId()`), and a `Reset()` method. The doc comment states the goal: in a later phase this is swapped for the EF Core `DbContext`.

```
public interface IDataSource
{
    List<User> Users { get; }
    List<Vehicle> Vehicles { get; }
    // ... Bookings, Payments, PromoCodes, Reviews, Notifications, PricingRules
    int NextUserId();
    int NextVehicleId();
    // ... one Next*() per entity
    void Reset();
}
```

4. InMemoryDataSource

The concrete implementation, registered as a **singleton** so all requests share one instance. Key characteristics:

Aspect	Detail
Storage	Plain List collections held in memory (lost on restart).
Id generation	Interlocked.Increment on private int sequences - atomic, no duplicates.
Thread safety	A private lock object; services also lock individual lists during reads/writes.
Seeding	SeedData() runs in the constructor, mirroring DbSeeder's data (ids set manually).
Reset()	Clears all lists and re-seeds - handy for tests/demos.

```
public int NextUserId() => Interlocked.Increment(ref _userIdSeq);

public void Reset()
{
    lock (_lock)
    {
        Users.Clear(); Vehicles.Clear(); /* ... */
        SeedData();
    }
}
```

Two stores, one caveat

Both stores are initialized at startup, but they are **not synchronized**. The business services operate exclusively on `InMemoryDataStore`; the EF Core `ApplicationDbContext` is set up and seeded but not yet used by the services. The intended final step is to migrate the services to read/write through EF Core (so data persists in Postgres) and then retire the in-memory store. The `IDataStore` abstraction exists to make that swap low-friction.

Migrations (related)

The Migrations/ folder holds `20260916163448_InitialSupabaseSetup`, which creates all eight tables in Postgres with the correct column types (e.g. `numeric(18,2)` for money, `timestamp with time zone` for dates) and identity primary keys. `ApplicationDbContextModelSnapshot.cs` records the current model so future migrations can diff against it.